ROS 2 on a 320 KiB Microcontroller

Interoperating with an NXP MR-CANHUBK344 over a single serial line, using nano-ros

Zephyr 4.4.0 · S32K344 Cortex-M7 · zenoh-pico · ROS 2 Humble
nano-ros `nros-v0.5.0-5172-g0a7397402`

What this is
A design description and a field report. The design half explains how a ROS 2 node is made to fit in 320 KiB with no MMU; the field half is what actually happened when it was brought up against a real ROS 2 graph. The two are interleaved on purpose — most of the engineering cost lived in the gap between them.

The honest summary: **the code was rarely the hard part.** Silent failures and invalid measurements were. Several sections below are accounts of being wrong, including twice reversing the same conclusion, because that is where the time went and it is the part that generalises.

Contents

1 What was built

1.1 The result, up front

A nano-ros node runs on the board and joins an ordinary ROS 2 graph through a `rmw_zenoh` router. The link is one UART at 115200 baud. No Ethernet, no IP stack in the image at all.

Capability	Status	Evidence
Publish / subscribe	**works**	`ros2 topic hz` 1.985–1.995 Hz; 552 messages across a five-minute soak, no session loss
Services	**works**	`ros2 service call /add_two_ints` returns `sum=7`, then `sum=99`
Graph introspection	**works**	`ros2 node list`, `topic list`, `service list`, `action list` all resolve
Action declaration	**works**	all four action keys byte-identical to a native `fibonacci_action_server`
Action goal completion	**fails**	goals do not complete; cause not isolated — see Section 4.2
Parameters	**fails**	`ros2 param list` returns nothing; services never registered — see Section 4.2

Table 1: State of ROS 2 interoperation on the MR-CANHUBK344.

Two of those rows are failures and they are on the first page deliberately. A report that buries them is not usable by anyone deciding whether to adopt this.

1.2 Why a microcontroller ROS node is a different problem

Constraint	Consequence
320 KiB SRAM, 128 KiB DTCM	Every buffer is a design decision. The final image sits at 91.1 % SRAM.
No MMU	No process isolation and no demand paging: a stack overrun corrupts a neighbour rather than faulting cleanly.
One UART, shared with nothing	The transport, and any debug output placed on it, contend for the same 87 µs-per-byte pipe. This turns out to matter enormously (Section 3.2).
No hardware entropy	Zephyr falls back to a timer-derived stand-in, which is deterministic from reset. This silently invalidated measurements for a long time (Section 3.3).
Cortex-M7, single core	Concurrency is preemption, not parallelism. Priority is the only lever, and it must actually be applied (Section 3.2).

Table 2: The constraints the rest of this document keeps returning to.

The board is an NXP MR-CANHUBK344: S32K344, Cortex-M7 at 160 MHz, 320 KiB SRAM plus 128 KiB DTCM, 4 MiB flash, running Zephyr 4.4.0.

2 Design

2.1 Layering

nano-ros separates four concerns. The value is in **where the seams are**, because each seam is what lets one of the four be replaced without touching the others.

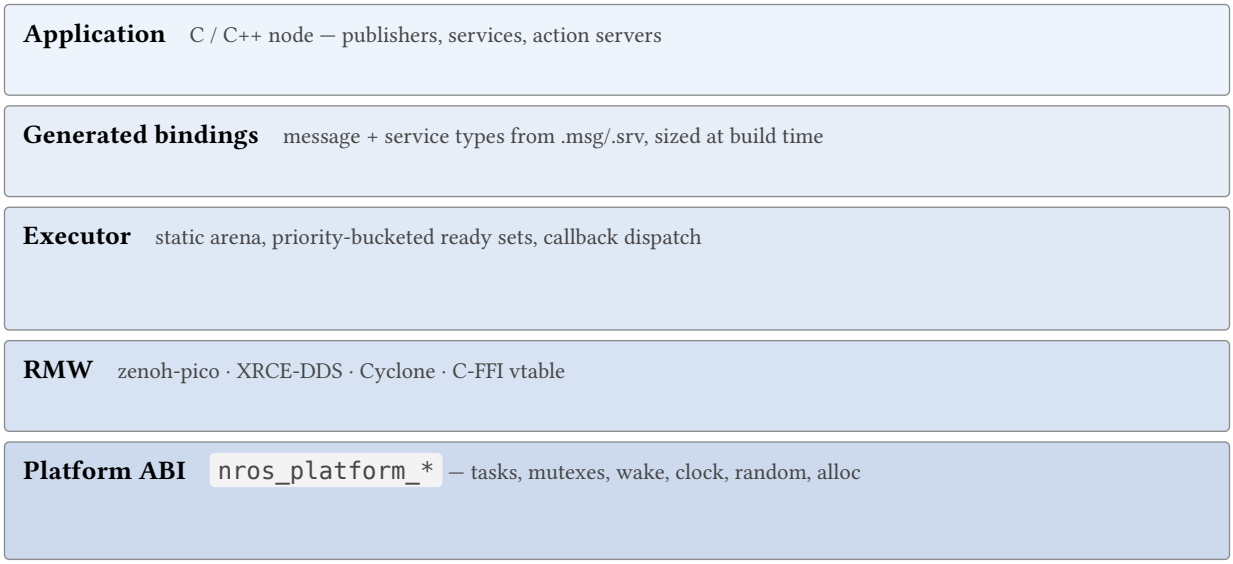

Figure 1: The four seams. Each layer names what the layer beneath must provide, and nothing more.

The platform ABI is the load-bearing one. It is a flat C interface — tasks, mutexes, a wake primitive, a clock, randomness, allocation — that each RTOS port implements. Two properties of it matter later:

Ask, do not assume a caller asks the port for sizes and capabilities rather than hard-coding them, because only the port knows (`nros_platform_wake_storage_size` and friends).

Attributes are honoured or refused, never silently dropped a port that cannot apply a requested stack size says so. *This rule was violated, and Section 3.2 is the consequence.*

2.2 Transport: zenoh-pico over a serial line

ROS 2 normally assumes a network. There is none here, so the board speaks zenoh-pico to a `rmw_zenoh` router running on the host, and the router bridges it into the ordinary ROS 2 graph.

The serial link has its own framing, which is not any protocol a network analyser knows:

`header`	`len`	`payload`	`crc32`
1 B	2 B, LE	`len` B	4 B

the whole frame COBS-encoded, `0x00` as delimiter

`header` carries the handshake flags `INIT` (`0x01`), `ACK` (`0x02`) and `RESET` (`0x04`). Session establishment is a four-message exchange:

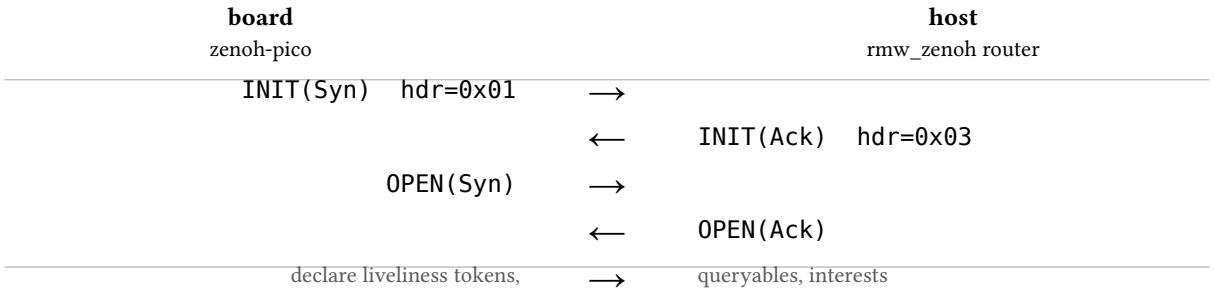

Figure 2: Serial session establishment, then entity declaration. Captured verbatim on the wire in Section 3.4.

After the handshake the board declares its entities: liveliness tokens for the node and each endpoint, plus queryables for services. The router keeps that state for one lease period (`Z_TRANSPORT_LEASE` = 10 s, expiry factor 3).

> **The key format is the interoperation contract**
> A nano-ros entity is visible to ROS 2 tooling only if its zenoh key matches byte-for-byte what `rmw_zenoh` would have produced:
>
> `@ros2_lv/<domain>/<zid>/0/<entity>/<NN|MP|SS>/%/<ns>/<node>/...`
>
> Getting this subtly wrong produces a node that is **present and silent** — it transmits happily and nothing ever answers. Section 3.1 is about the two ways we got it wrong.

2.3 Memory: static by default

The executor's callback arena is sized at build time from the declared entity counts, not grown at runtime. Buffers that could be large are pushed behind an explicit allocation funnel (`nros_platform_alloc`) so that a bare-metal image can account for — or forbid — every byte.

Region	Used	Of
Flash (`text` + `data`)	342 488 B	4 144 896 B (8.3 %)
SRAM (`data` + `bss`)	298 656 B	327 680 B (91.1 %)
DTCM	0 B	131 072 B

Table 3: Footprint of the action-server image, from a CLEAN build. An incremental build under-reported SRAM by 15 KiB, carrying stale smaller artifacts; only `rm -rf` on the west build directory gives a trustworthy figure.

One finding is worth repeating because it generalises past this board: an unused `malloc` arena was still reserving **24 556 B** of SRAM. The allocator itself had been garbage-collected by the linker — nothing called it — but `CONFIG_COMMON_LIBC_MALLOC_ARENA_SIZE` reserves its pool independently of whether any code survives to use it. Setting it to `0` moved the image from 83.3 % to 76.2 % SRAM at the time. **Dead code is collected; dead *reservations* are not.**

2.4 The executor, and a known weakness

The executor wakes on a semaphore signalled from the transport's arrival path — sub-millisecond, callable from an ISR — and then walks every registered entity asking `has_data`, up to `MAX_CALLBACK_SLOTS` = 64 indirect calls, on every spin regardless of what woke it.

Property	Today
Wake latency	one `k_sem_give`, sub-millisecond, ISR-safe
Wake precision	**binary** — "something happened"
Finding out what	**O(N) indirect calls**, every spin

Table 4: The executor is woken precisely and then searches blindly.

The arrival path knows exactly which entity received data; the callback it fires takes only an executor-wide context pointer, so that identity is discarded and rediscovered by exhaustive search. The destination already exists — the ready set is a `u64` bitmap whose pop is a single `CLZ`-class instruction — so the missing piece is only that nothing but the scan writes into it.

This is written up as a proposal (RFC-0084) with an implementation plan (phase-396): let the wake carry an entity token, `fetch_or` it into an atomic bitmap, and keep the scan as the fallback for backends that cannot

attribute an arrival. It is **not** implemented. It is included here because a design description that omits its own known weaknesses is marketing.

3 Practical experience

3.1 Naming is the whole battle

Two independent naming faults, both producing the same symptom: a node that appears in the graph, transmits, and is never answered.

Domain missing from the key. Services were declared without the domain identifier. Seven call sites constructed a `ServiceInfo` and every one of them omitted `.with_domain()`. The node was on domain 10; its services announced themselves on domain 0. Everything looked healthy from the board.

Type names not DDS-mangled. `rmw_zenoh` writes type names in DDS form. A service typed `example_interfaces/srv/AddTwoInts` must appear on the wire as:

```
example_interfaces::srv::dds_::AddTwoInts_
```

Note both the `dds_` infix and the trailing underscore. We produce this with an allocation-free `Display` wrapper so the mangling happens inside `format_args!` rather than into a temporary buffer — on a part with 320 KiB, a formatting allocation on a declaration path is worth avoiding.

After both fixes, `ros2 service call` returns `sum=7`, and the action keys are byte-identical to a native server's.

> **A non-bug that looks exactly like a bug**
> `ros2 service list` shows none of the board's action services. This is correct: `_action/` services are hidden unless `--include-hidden-services` is passed, and with it all three appear, properly named. Time was spent on this. The lesson recurs in Section 4.2 — an empty listing and a broken node are indistinguishable from the outside.

3.2 The starvation story

This is the deepest defect found, and its shape is more interesting than its fix.

The symptom. Actions failed while publish/subscribe was fine. The board's session expired at exactly twice the transport lease. Suspicion fell on the router's keepalives, and stayed there for a long time — through six successive hypotheses, an instrumented build of the router from source, and a filed issue that turned out to be wrong in every particular. The router was innocent: its timer fired, its keepalive arm fired, its writes succeeded, and the frames it emitted were well formed.

The actual cause. The receiver was dropping bytes, and could not report it.

The zenoh read task is created with a declared scheduling priority. That priority is wired through Kconfig, through CMake, and into the task attributes — and is then discarded at three separate layers:

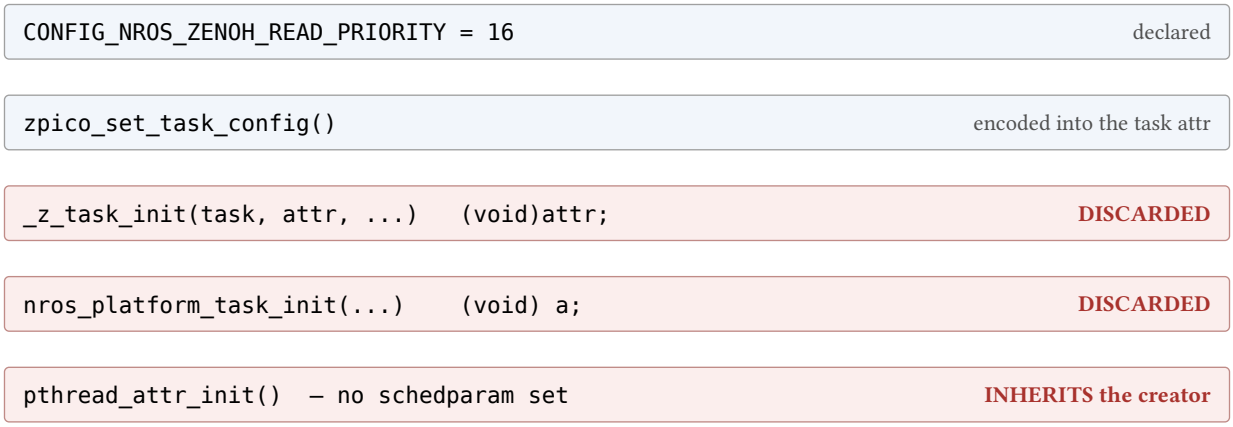

Figure 3: A declared attribute, accepted by every layer and applied by none.

Because no layer set `PTHREAD_EXPLICIT_SCHED`, Zephyr's default of `PTHREAD_INHERIT_SCHED` applied and the read task was born at the priority of the thread that created it — the executor. Equal priority, on a kernel with timeslicing enabled:

`CONFIG_TIMESLICE_SIZE` = 20 ms · 115200 baud ⟶ 87 µs/byte

20 ms ÷ 87 µs ≈ **230 bytes** arriving while the reader is not scheduled

against an LPUART RX FIFO a few entries deep

The read loop's `k_yield()` handed the executor a full timeslice on every poll miss. Overrun is arithmetic, not a race. And it was invisible: `uart_poll_in` reports "no character available" but never "a character was destroyed before you asked", because nothing called `uart_err_check`. Adding that call produced the proof immediately — the overrun flag set on exactly the truncated frame and nowhere else.

This also explains the load dependence that made it look like an action bug: at idle the executor is blocked in its wake wait and the reader is the only runnable thread, so the handshake survives; under three queryables and two publishers the executor is runnable and each `k_yield()` costs 20 ms.

The obvious fix would have been worse

The natural correction is "give the reader a real-time priority", i.e. `SCHED_FIFO`. On Zephyr that is wrong. Its POSIX layer maps the two policies onto its two priority bands:

Policy	Band	Ceiling
`SCHED_FIFO`	cooperative	`CONFIG_NUM_COOP_PRIORITIES` − 1
`SCHED_RR` / `SCHED_OTHER`	preemptible	`CONFIG_NUM_PREEMPT_PRIORITIES` − 1

A cooperative thread is never preempted, and this one busy-polls a UART. It would have traded a 20 ms starvation of the reader for an unbounded starvation of everything else. `SCHED_RR` is the correct choice — and the fact that the same constant means something different here than on Linux is exactly why the priority map belongs in the per-port layer.

Honouring the priority was still not sufficient by itself. `SCHED_RR` maps as `zephyr = NUM_PREEMPT − posix − 1`, so the top preemptible slot is Zephyr priority 0 — which is where `CONFIG_MAIN_THREAD_PRIORITY` already put the executor. The reader cannot be placed above a thread already holding the top slot, so main was moved down to 5.

3.3 The confound

Having fixed the above, the next step was to compare polled reception against an interrupt-driven receiver. That comparison was invalid, and so was a conclusion already written down from it.

The board drew the same zenoh identity on every boot.

```
boot 1 zid: 1322740661b45746fa29b1803f32f5eb
boot 2 zid: 1322740661b45746fa29b1803f32f5eb
boot 3 zid: 1322740661b45746fa29b1803f32f5eb
```

Identical across resets, reflashes and power cycles. With no hardware entropy configured, Zephyr backs `sys_rand32_get` with a timer reading, and the path from reset to the point zenoh-pico draws its identity is deterministic — so the reading is the same number every time.

A zenoh id is the identity a router holds a session's state under. A board that reboots into the same one is, to the router, the peer it already has. Every measurement that crossed a reconnect therefore depended on **what the router remembered**, not on the firmware under test.

The fix seeds a small PRNG from a `.noinit` word carried across reset — Zephyr does not zero that section, so it holds the previous boot's state on a warm reset and SRAM power-up noise on a cold one — mixed with cycle and uptime counters. It is substituted **only** when there is no real entropy source, because a PRNG must not shadow good entropy, and it is documented in the source as not being a CSPRNG: it makes an identity unique, it does not make a secret unguessable. Cost: 24 bytes of RAM.

```
boot 1 zid: 06f6a004dc9321cfda5fb1359a10e61e
boot 2 zid: ac181f9c54a245950bca25d7ee814faa
boot 3 zid: 9b1311b02b0e4c6a64fb5544281b75dc
```

> **The part worth admitting**
> Under the confound, the interrupt-driven receiver appeared to break the board's declarations. That was written up and committed. Removing the confound showed it was false — and it had already been correctly identified as a confound once, then **over-corrected** on the strength of a run whose router had silently failed to start (`Address already in use`) while a stale process still owned the serial port.
>
> So the same conclusion was reversed twice before the evidence was clean. What fixed it was not more thinking: it was a fixed harness that kills every port holder by PID, waits for the router to announce itself, resets the board exactly once, and only then measures — plus an unrelated host node in the graph as a live control, so "the board is missing" could be distinguished from "the router is broken".
>
> A measured claim that has been committed to a repository and later fails to reproduce must be withdrawn in the same place it was made. Two were.

3.4 Tooling: what exists, what had to be built

Wireshark does not help here, for two independent reasons. Its Zenoh dissector arrived in 4.2 (the host had 3.6.2), and even where it exists it dissects Zenoh over TCP/UDP. Our transport is a raw UART carrying the COBS framing of §2.2 — there is no layer for the dissector to attach to.

A serial tap. `socat` inserts a pseudo-terminal between the router and the real device and dumps both directions; a small decoder undoes COBS and prints frames:

```
socat -x -v /dev/ttyUSB0,raw,echo=0,b115200 PTY,link=/tmp/ttyTAP,raw,echo=0 2>/tmp/
tap.hex
# point the router at  serial//tmp/ttyTAP  instead of /dev/ttyUSB0
./serial-tap.py /tmp/tap.hex
```

```
[1] board->router hdr=0x01(INIT)     len=0
[2] router->board hdr=0x03(INIT,ACK) len=0
[7] board->router hdr=0x00 len=67   @ros2_lv/10/ac181f9c54a24...
[9] board->router hdr=0x00 len=112  10/fibonacci/_action/send...
```

Two details cost time and are worth passing on. `socat`'s per-transfer header lines contain byte-shaped tokens — a timestamp like `2026/08/28 20:17` yields `20`, `28`, `17` — so bytes must be taken only from the fixed-width hex column; parsing the whole file corrupts every frame. And the trailing four bytes are **reported, not verified**: no standard CRC-32 variant tried (ISO-HDLC, BZIP2, MPEG-2, POSIX, JAMCRC, CRC-32C, AUTOSAR, CD-ROM), over either the payload or header-plus-payload, reproduces the observed value. Printing `CRC-BAD` would have meant "I do not know the algorithm" while reading as corruption on every good frame.

Out-of-band board output. SEGGER RTT is the right channel precisely because it does not touch the link under test:

```
pyocd rtt -t s32k344 -a 0x20404010      # address from: nm zephyr.elf | grep
_SEGGER_RTT
```

It needs a pseudo-terminal (`script -qec ...`) or it fails with `Inappropriate ioctl for device`. With zenoh-pico's own debug level compiled in, this is how we know the board completes its handshake, registers every entity, and then goes silent.

The pair is what the remaining investigation needs: the tap says whether a request reached the wire and in which direction; RTT says whether the application layer ever saw it.

4 Reflection

4.1 Patterns worth generalising

A silent failure costs more than a loud one, by orders of magnitude. The UART overrun had no error path. That single missing `uart_err_check` turned a one-line diagnosis into six wrong hypotheses, an instrumented build of somebody else's router, and a filed issue against an innocent component. Instrumentation that is only added while debugging is added too late.

An attribute that is accepted and then dropped is worse than one that is refused. Every layer in Figure 3 took the priority and returned success. A port that cannot honour a request must say so; silence converts a configuration error into a performance mystery.

The same failure mode recurs at the interface. An empty `ros2 param list` (Section 4.2) is indistinguishable from a node failing to answer. Hidden `_action/` services look like missing ones. In each case the system was behaving correctly and the **observer** could not tell.

Every A/B needs a live control. Without an unrelated host node in the graph, "the board is missing" and "the router is broken" produce identical output. Adding one control turned an irreproducible argument into a two-line answer.

Prefer a fixed harness to a careful hand-run sequence. Numbers gathered by hand did not reproduce under a scripted protocol. The scripted one kills port holders by PID, waits for an explicit readiness line, and resets exactly once. Hand-running was the source of the bad numbers, not the system under test.

Beware `pkill -f` on a shared machine. It matches the command line of the shell running it. This killed the working session twice. Match by PID.

4.2 Still open

Action goals do not complete. Both receive paths behave identically here, so the receive path is **not** currently implicated. The next step is instrumenting the goal path itself: the tap shows whether the request frame reaches the wire, RTT whether the application layer sees it.

No link-level resync on serial. Resetting the board while the router holds the link puts the router into a repeating `Unexpected Init flag in message` from which it does not recover; the router must be restarted after a board reset. This survives the identity fix and is a genuine limitation of the link.

No hardware entropy. The PRNG of Section 3.3 makes identities unique. It is not a security answer, and the production gap is unchanged.

Parameter services are never registered. `register_parameter_services()` is opt-in — correctly, since six service servers cost RAM — but no example in the tree calls it, so every nano-ros node presents to standard tooling as one whose parameter interface is broken rather than one that declined to have it.

Readiness is scanned, not pushed. Figure 1 describes the executor's O(N) scan; RFC-0084 and phase-396 describe the fix. Unimplemented.

4.3 What we would do differently

Instrument the layer you do not own **before** you need it. The overrun check, the identity print, and the serial tap were each built in response to a specific failure; all three would have paid for themselves on day one, and two of them are three lines of code.

Fix the measurement apparatus before trusting a measurement. The identity problem was known and recorded as a "production gap" long before it was understood to be actively corrupting results. A stand-in labelled `CONFIG_TEST_RANDOM_GENERATOR` was doing exactly what its name says, in a context where that was not acceptable.

And write down the wrong turns. Three of the sections above are more useful for the hypotheses they eliminate than for the fix they land.

5 Appendix: configuration that matters

```
# transport
CONFIG_NROS_ZENOH_LINK_SERIAL=y
CONFIG_NROS_ZENOH_LOCATOR="serial/uart@40330000#baudrate=115200"
CONFIG_NROS_ZENOH_MULTI_THREAD=y

# no IP stack in a serial-only image
CONFIG_NETWORKING=n
CONFIG_NET_SOCKETS=n
CONFIG_NET_L2_ETHERNET=n

# reclaim the arena of an allocator nothing calls  (-24 556 B)
CONFIG_COMMON_LIBC_MALLOC_ARENA_SIZE=0

# the transport reader must outrank the executor  (§3.2)
CONFIG_MAIN_THREAD_PRIORITY=5
CONFIG_NROS_ZENOH_READ_PRIORITY=16

# stacks and slots
CONFIG_MAIN_STACK_SIZE=16384
CONFIG_NROS_ZEPHYR_TASK_SLOTS=6

# out-of-band diagnostics — NOT on the UART under test
CONFIG_USE_SEGGER_RTT=y
CONFIG_RTT_CONSOLE=y
CONFIG_UART_CONSOLE=n
CONFIG_LOG_MODE_IMMEDIATE=y
```

6 Appendix: reproducing the bring-up

Order matters. The router must be listening before the board's first `INIT`, and the board must not be reset while the router holds the link.

```
# 1. clear any previous holder — by PID, never `pkill -f`
for p in $(ss -lntp | grep 7447 | grep -oE 'pid=[0-9]+' | cut -d= -f2 | sort -u); do
kill -9 $p; done
for p in $(fuser /dev/ttyUSB0 2>/dev/null); do kill -9 $p; done

# 2. router first, and wait for it to announce itself
ZENOH_ROUTER_CONFIG_URI=router-serial.json5 \
ZENOH_CONFIG_OVERRIDE='listen/endpoints=["tcp/[::]:7447","serial//dev/
ttyUSB0#baudrate=115200"]' \
ROS_DOMAIN_ID=10 ros2 run rmw_zenoh_cpp rmw_zenohd &

# 3. only then, reset the board exactly once
pyocd reset -t s32k344

# 4. measure
ROS_DOMAIN_ID=10 ros2 node list
```

The TCP endpoint is not optional: `ZENOH_CONFIG_OVERRIDE` **replaces** the endpoint list rather than adding to it, so a serial-only override silently removes the listener that local ROS 2 tooling connects to.

7 Appendix: issue index

ID	Subject	State
0821	Fault at twice the lease was `main`'s stack, not the transport	fixed
0822	zenoh-pico Zephyr thread-stack slots leak across reconnects	fixed, upstream branch
0824	Services declared on domain 0 with unmangled type names	fixed
0839	Action image session expires every 20 s	open
0848	Filed against the router; wrong in every particular	withdrawn
0852	Declared read-task priority discarded at three layers	fixed
0864	Identical zenoh id on every boot	fixed
0865	No example registers parameter services	open
RFC-0084	Readiness is pushed by the arrival path, not scanned	draft
phase-396	Implementation plan for RFC-0084	not started